

Course code and name:	B37VB – Praxis Programming
Type of assessment:	Individual
Coursework Title:	Game Project - Paint
Student Name:	Luan Tran
Student ID Number:	H00468889

Declaration of authorship. By signing this form:

- **I declare** that the work I have submitted for individual assessment OR the work I have contributed to a group assessment, is entirely my own. I have NOT taken the ideas, writings or inventions of another person and used these as if they were my own. My submission or my contribution to a group submission is expressed in my own words. Any uses made within this work of the ideas, writings or inventions of others, or of any existing sources of information (books, journals, websites, etc.) are properly acknowledged and listed in the references and/or acknowledgements section.
- I confirm that I have read, understood and followed the University's Regulations on plagiarism as published on the [University's website](#), and that I am aware of the penalties that I will face should I not adhere to the University Regulations.
- I confirm that I have read, understood and avoided the different types of plagiarism explained in the University guidance on [Academic Integrity and Plagiarism](#)

Student Signature (*type your name*): Luan

Date: 01/04/2025

Contents

Introduction.....	2
Implementation	2
App.....	3
Canvas.....	3
Render	3
Shader	3
User Interaction and Controls	3
Flow chart	4

DUE TO THE LIMITATION OF 5 PAGES seek the src file provided in the GitHub repository for reference:

<https://github.com/loggingtruck/paint>

Introduction

This project is a simple 32×32 black-and-white pixel canvas paint application built using C, along with API like OpenGL, Libraries like GLFW, and GLM. It provides a minimalistic environment for creating pixel art, where users can toggle pixels with mouse clicks. The application is lightweight, ensuring smooth performance even on low-end hardware.

Implementation

The application is built in C and utilises the following libraries:

- OpenGL for rendering vertices, shaders, canvas, pixel, grid
- GLFW for window management and input handling
- GLM for mathematical calculations

App

The App struct handles window creation and OpenGL context initialisation. It uses GLFW to set up an 800x800 window with an OpenGL 3.3. Window dimensions are stored globally, and the viewport is updated dynamically when the window is resized.

Locate to **App.c** within **src** folder for reference code

Canvas

The Canvas struct maintains a 32×32 grid where each pixel is represented by a value indicating its colour (0 = white, 1 = black). It initializes all pixels to white by default.

Users can modify pixels via the `canvas_set_pixel` function, which updates the pixel data from 0 to 1 or vice versa.

Locate to **Canvas.c** within **src** folder for reference code

Render

Rendering is handled by the Renderer struct. It initializes shaders and buffers necessary to draw the pixel grid. The shader program is created using a vertex and fragment shader, enabling pixel rendering. The screen is cleared to a white background each frame, and only black pixels are drawn, reducing unnecessary computations.

Locate to **Rendering.c** within **src** folder for reference code

Shader

The Shader struct manages OpenGL shaders. It compiles, links, and applies shaders dynamically. Uniform variables control pixel colour and positioning, ensuring that the grid remains aligned regardless of window resizing.

Locate to **Shader.c** within **src** folder for reference code

User Interaction and Controls

- Mouse input
 - Left click to turn a pixel black
 - Right click to turn a pixel white
- Keyboard input
 - ESC key to exit the application

Flow chart

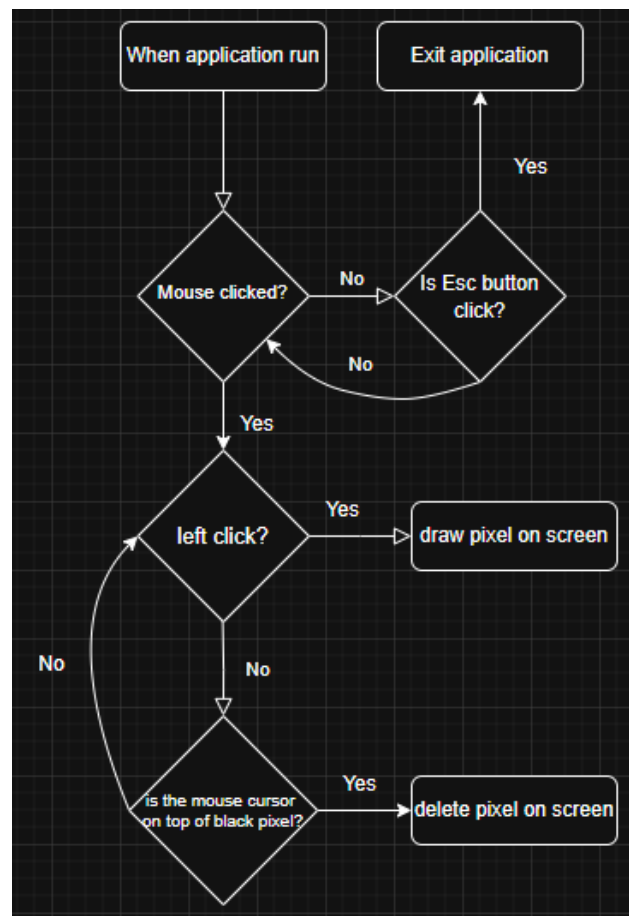


Figure 1. user interaction

